ONE COMPREHENSIVE CHEAT SHEET

Large Language Models, explained for engineers.

Quick conceptual intuition, the essential mathematics, complexity, trade-offs, and production decision rules — in one searchable PDF.

MASTER INDEX · click any section to jump there

02 An LLM in one page
03 Tokens, embeddings & positions
04 Self-attention: intuition → math
05 Attention variants & RoPE
06 Transformer blocks & model families
07 Pretraining data & objectives
08 Optimization & training math
09 Post-training & alignment
10 PEFT, LoRA & QLoRA
11 Sampling and decoding
12 Prefill, decode & the KV cache
13 Quantization & acceleration
14 RAG vs long context vs fine-tuning
15 Chunking, embeddings & indexing
16 Reranking & advanced retrieval
17 Tools, agents & MCP
18 Evaluating the language model
19 RAG & agent evaluation
20 Serving architecture & request flow
21 Security, reliability & governance
22 What to change when something fails
23 Formula & complexity crib
24 Sources & navigation keys

READ IT TWO WAYS
Concepts → equations → engineering decisions
Use as a daily reference, review guide, or interview refresher.

Source-backed synthesis
9 reference texts · 24 pages

Mohammed Vasim

in /in/mohd-vasim

An LLM in one page

From training tokens to a production answer

CONCEPTUAL

Mental model

Core idea. A language model estimates a distribution over the next token, conditioned on prior tokens. Repeated sampling yields text.

Not a database. Parameter knowledge is statistical; factual answers need validation or external evidence.

Task adaptation. Pretraining builds general capability; post-training shapes instruction-following and behavior.

SYSTEMS

Two distinct workflows

Track	Pipeline
Model creation	Clean corpus → tokenize → pretrain → post-train → evaluate → register
Application delivery	Define task → select model → prompts/RAG/tools → evaluate → serve → monitor

> Not every application needs custom training. Start with a strong baseline before adding complexity.

A compact vocabulary

Term	Meaning
Token	Unit processed/generated by the model; may be a subword or byte sequence.
Parameter	Learned numerical weight in the network.
Embedding	Learned or computed vector representation.
Context window	Maximum supported input + generated context, subject to API/model limits.
Checkpoint	Saved parameter state (and, during training, often optimizer state).

MECHANICS

Inference path

User request → prompt/template → tokenize → prefill → decode → validate/output

With RAG: retrieve evidence before prompt assembly. **With agents:** the application loops through model decisions and bounded tool executions.

DECISION RULE

Choose the lightest adaptation

Need	Preferred first approach
Prompt + examples	Shape task behavior; fastest baseline.
RAG / tools	Supply current/private facts or execute real actions.
SFT / LoRA	Teach stable formats, behavior, domain patterns.
Pretraining	Only when data, budget, and capability gap justify it.

What “good” really means

- **Quality:** correctness, helpfulness, groundedness, robustness.
- **System:** p50/p95 latency, tokens/sec, throughput, uptime.
- **Business:** cost per successful task, escalation rate, time saved.
- **Trust:** privacy, access control, prompt-injection resistance, auditability.

> Evaluate end-to-end behavior on real tasks, not only model leaderboard scores.

Mohammed Vasim

in /in/mohd-vasim

Tokens, embeddings & positions

What the network actually consumes

CONCEPTUAL

Tokenization

What. Maps text to discrete IDs: text → tokenizer → token IDs. The tokenizer is model-specific.

Common schemes. BPE, WordPiece, unigram; byte-level fallbacks improve coverage.

Why care. Language, whitespace, code, and formatting change token counts and costs.

ENGINEERING

Vocabulary & special tokens

- **BOS/EOS:** sequence boundaries.
- **PAD:** equal-length batching; ignored by masks when appropriate.
- **Role separators:** system/user/assistant and tool message framing in chat templates.
- **Unknown text:** tokenizer-specific handling; byte-based tokenizers can cover arbitrary input.

TECHNICAL

Embeddings

Input: token index `i` looks up row `E[i]` in an embedding table.

Shape: embedding matrix `V × d_model`; a length-`n` sequence becomes `n × d_model`.

Output: hidden states ultimately project to vocabulary logits; input/output matrices may share weights.

X = Embedding[token_ids]

ARCHITECTURE

Position information

Method	Mechanism
Absolute position embedding	Adds learned or fixed position vectors to token embeddings.
RoPE	Rotates paired query/key dimensions as a function of position.
Relative / bias schemes	Encode relative distance directly in attention scores.

Trade-off: length extrapolation and long-context behavior are implementation- and training-dependent.

PRODUCTION

Context budgeting

Window accounting: instructions + chat history + tool output + retrieved chunks + output reserve must fit.

Symptoms: truncation, missing evidence, degraded long-range retrieval.

Fixes: compress history, select evidence, cap tool payloads, use prompt caching where supported.

Representation pitfalls

- An embedding is a numeric representation, not guaranteed human-interpretable meaning.
- Similarity of vector embeddings depends on the model, normalization, domain, and query type.
- Never compare token counts across tokenizers as if they were the same unit.
- Keep tokenizer, chat template, and model version aligned.

> Example: “unbelievable” may be one token or several. Token count ≠ word count.

Mohammed Vasim

/in/mohd-vasim

Self-attention: intuition → math

The core operation behind context mixing

CONCEPTUAL

Query / key / value

Query: what a token seeks; **key:** how each token can be matched; **value:** what information it contributes.

Each is a learned projection of hidden states `X`:

$$Q = XW_Q \quad K = XW_K \quad V = XW_V$$

Tokens attend to other allowed tokens; causal decoder attention masks future positions.

FORMULA

Scaled dot-product attention

$$A = \text{softmax}(QK^T / \sqrt{d_k} + M)$$

$$\text{Attention}(Q,K,V) = A\,V$$

QK^T scores relationships; $\sqrt{d_k}$ scales logits; `M` adds mask/bias; softmax normalizes weights.

Mask: disallowed keys receive effectively $-\infty$ before softmax.

Dimensions (single head)

Tensor	Shape
X	n × d_model
Q, K	n × d_k
V	n × d_v
QK^T / √d_k	n × n
Output	n × d_v

> Batch and attention-head axes are omitted here for clarity.

MECHANICS

Why multiple heads?

Independent heads learn distinct projection subspaces; concatenate head outputs and apply `W_O`.

$$\text{MHA}(X) = \text{Concat}(\text{head}_1,\ldots,\text{head}_h)W_O$$

Typical head dimension: `d_head = d_model / h` for equally sized heads.

COMPLEXITY

Compute and memory costs

Item	Asymptotic cost
Dense score construction	$O(n^2\ d_k)$ per head; more generally $O(n^2\ d_{model})$ across heads.
Attention matrix storage	$O(h\ n^2)$ scores per batch element if materialized.
Q/K/V & projections	Typically $O(n\ d_{model}^2)$ projection work.

> FlashAttention changes attention-matrix *memory traffic/storage*, not the quadratic arithmetic of full dense attention.

Worked intuition

In “The cat that chased the mouse **was** hungry”, the representation at “was” may put weight on “cat”.

Attention weights show routing of information; they are not a complete explanation of model decisions.

In causal generation, the token at position `t` can attend only to positions ≤ `t`.

Mohammed Vasim

/in/mohd-vasim

Attention variants & RoPE

What changes memory, speed, and context behavior

COMPARISON

MHA vs GQA vs MQA

Variant	Layout	Trade-off
MHA	Each query head has its own K/V head.	Higher KV-cache footprint.
GQA	Groups of Q heads share K/V heads.	Less cache/bandwidth, common compromise.
MQA	All Q heads share one K/V head.	Smallest KV cache; possible quality trade-off.

Invariant: query heads need not equal KV heads for grouped/multi-query attention.

MEMORY

KV-cache sizing

$$\text{KV bytes} \approx 2 \cdot L \cdot B \cdot T \cdot H_{kv} \cdot d_{head} \cdot b$$

L layers, B batch/sequences, T stored tokens, H_{kv} KV heads, b bytes per element.

Example: 32 layers × 1 seq × 4k tokens × 8 KV heads × 128 dim × 2 bytes × 2(K,V) ≈ **512 MiB**.

Ignores metadata, fragmentation, and other activations.

MECHANICS

RoPE (rotary position embeddings)

Rotates pairs of query/key vector coordinates by position-dependent angles before dot products. Relative offsets affect attention scores.

Benefit: position-aware attention without adding absolute position vectors to embeddings.

Caution: scaling/interpolation for longer contexts must be validated; advertised context length alone does not ensure accuracy.

LONG CONTEXT

Windowed and sparse attention

- **Sliding window:** each token attends to nearby tokens; compute can approach $O(n \cdot w \cdot d)$ for window size w.
- **Global/selected tokens:** add far-range links for special tokens or summaries.
- **Hybrid:** combine full and local layers.
- **Trade-off:** improved scaling but constrained direct access to distant tokens.

Use this decision rule

Long prompts limited by KV memory? Consider fewer KV heads, cache quantization, paging, context management.

Long prompts limited by prefill? Reduce redundant tokens, leverage efficient kernels or caching.

Missing distant facts? Benchmark actual tasks; try retrieval and reranking, not context expansion alone.

Mohammed Vasim

/in/mohd-vasim

Transformer blocks & model families

How layers become language models

ANATOMY

A decoder block

Hidden state → norm → causal attention → residual → norm → MLP → residual

Residual: preserve/add earlier representation, easing optimization.

Normalization: LayerNorm or RMSNorm stabilizes activations.

MLP: per-token nonlinear transformation; often gated (e.g., SwiGLU).

Exact order differs: pre-norm/post-norm and parallel variants exist.

ARCHITECTURE

Families

Type	Typical use
Encoder-only	Bidirectional self-attention; classification/embedding often natural.
Decoder-only	Causal generation; common modern text LLM design.
Encoder–decoder	Encode source then cross-attend while generating target.

> "Decoder-only" describes architecture, not a restriction to chatbot use.

TRADE-OFF

Dense vs mixture of experts

Dense: most network parameters participate for each token.

MoE: router selects a subset of expert feed-forward networks per token (often top-k).

Why: more total capacity without proportional active compute.

Cost: routing, load balance, communication, and memory for all resident experts.

FORMULA

Language modeling head

$$\text{logits}_t = h_t W_{\text{vocab}}^T$$

$$p(x_{t+1} \mid x_{\le t}) = \text{softmax}(\text{logits}_t)$$

The hidden state at the final prompt position predicts the next token; append it and repeat for autoregressive generation.

SIZING

Parameters ≠ runtime cost

Parameter count affects memory and matmul cost, but actual latency depends on active parameters (MoE), sequence length, batching, KV layout, kernels, and hardware.

Rough dense decoder terms: attention projections scale with $\sim 4d^2$ weights per layer, MLP with $\sim 2d \cdot d_{ff}$ (ungated), excluding norms/biases.

Model selection checklist

- Task fit: text, multilingual, code, vision, structured output.
- License and data-use restrictions.
- Benchmarks on *your* data, not marketing examples.
- Context and tool-calling reliability.
- Deployment fit: weights, VRAM/RAM, throughput, token price.

> Bigger is not automatically better for a specific production workflow.

Mohammed Vasim

/in/mohd-vasim

Pretraining data & objectives

The information pipeline before optimization

LIFECYCLE

Data preparation

Sources → permissions → cleaning → de-dup → quality filters → tokenize → packing

Quality levers: coverage, deduplication, formatting, contamination avoidance, language/domain mix.

Data governance: respect consent, licenses, personal data, and retention requirements.

OBJECTIVE

Causal language modeling

At each position predict the next token from the preceding context. Shift labels by one token for training.

$$L_{CE} = -\frac{1}{N} \sum_t \log p_\theta(x_t \mid x_{<t})$$

Training uses teacher forcing: the ground-truth previous tokens are available; inference uses its own generated tokens.

Other objectives

Objective	Typical application
Masked modeling	Infer hidden tokens from bidirectional context.
Span corruption / seq2seq	Reconstruct missing spans from an encoded input.
Contrastive representations	Bring related embeddings together and unrelated apart.

IMPLEMENTATION

Packing and masking

Packing: join shorter sequences into fixed-length training blocks to limit padding waste.

Attention mask: prevent cross-example leakage if samples should be independent.

Loss mask: exclude padding and often prompt-only regions in instruction tuning.

Sequence length: longer sequences raise activation and attention expense.

EVALUATION

Dataset splits & leakage

- Split by source/time/entity when near-duplicates may leak across sets.
- Remove benchmark/test contamination before reporting capability.
- Audit language and domain coverage; track source provenance.
- Measure data quality and downstream task impact, not only corpus size.

DECISION

When pretraining is justified

Rarely the first step for a product. Consider continued pretraining if a large domain corpus and persistent vocabulary/knowledge gap justify the cost.

Use prompts, retrieval, and supervised fine-tuning first when they solve the actual issue.

Mohammed Vasim

/in/mohd-vasim

Optimization & training math

The quantities worth memorizing

WORKFLOW

Training loop

Batch → forward → loss → backward → clip/update → log/validate

Gradient accumulation: simulates a larger effective batch via several microbatches.

Effective global batch: microbatch × accumulation × number of data-parallel workers.

FORMULA

Gradient descent & AdamW

$$\theta \leftarrow \theta - \eta \cdot \nabla_\theta L$$

η learning rate; AdamW uses moving averages of first/second moments plus decoupled weight decay.

Common controls: warmup/schedule, gradient clipping, weight decay, checkpoint interval, validation cadence.

Training stability signals

Signal	First investigation
Loss spikes / NaNs	Check LR, precision, scaling, data anomalies, gradients.
Train↓ / val↑	Overfitting; review data, regularization, early stopping.
Slow tokens/sec	Investigate input pipeline, padding, network, utilization.
Unstable fine-tune	Check mask, chat template, batch size, optimizer.

SYSTEMS

Memory anatomy

Training memory = weights + gradients + optimizer state + activations + framework buffers.

Mixed precision: FP16/BF16 accelerates compatible ops; optimizer state may remain higher precision.

Checkpointing: recomputes activations during backward to reduce activation memory at extra compute.

ZeRO / FSDP: shard training state across devices; communication and complexity increase.

TRADE-OFF

Scaling concepts

- **Data parallelism:** replicate model; shard batches and synchronize gradients.
- **Tensor parallelism:** split layer computation and collectives across devices.
- **Pipeline parallelism:** distribute layers across stages; manage bubbles.
- **Compute-optimal training:** balance parameter count and training tokens under a fixed budget.
- **Hardware efficiency:** high utilization still needs accurate quality validation.

What to checkpoint

Model weights, tokenizer/config, optimizer + scheduler, RNG state, step/counters, and data progress when exact resume matters.

Version datasets and training code; record experiment lineage.

Mohammed Vasim

/in/mohd-vasim

Post-training & alignment

Turning a base model into an assistant

PIPELINE

Stage sequence

Pretrained model → SFT → preference optimization → safety/task eval

SFT teaches supervised demonstrations. Preference tuning shifts relative response desirability. Neither guarantees factual correctness.

TECHNICAL

Supervised fine-tuning (SFT)

Data: instruction/input and desired assistant response, usually serialized in the model chat template.

Loss: next-token cross entropy, often computed only over assistant target spans.

$$L_SFT = -\Sigma_{t \in assistant} \log p\theta(y_t \mid x, y_{<t})$$

When: formatting, instruction-following, consistent style, tool-call patterns.

Preference learning

Method	Main idea
RLHF	Fit a reward/preference signal, then optimize policy using RL; complex, tunable.
DPO	Directly optimize preference pairs without a separate online RL loop in standard form.
Other offline preference methods	Alternative objectives and regularization with similar chosen/rejected data.

MECHANICS

DPO intuition

For each prompt, increase relative preference for chosen y_+ over rejected y_- compared with a reference policy.

Key hyperparameter: β governs preference strength relative to the reference.

Failure mode: noisy or biased preference pairs can reinforce unwanted behaviors.

QUALITY

Alignment failure modes

- **Over-refusal:** harmless requests are rejected.
- **Reward hacking:** optimize proxy reward, not true user outcome.
- **Format compliance without correctness:** schema valid, content wrong.
- **Capability regression:** new behavior degrades older tasks.
- **Preference bias:** annotator criteria do not match deployment needs.

Before training

> First test: clear instructions + few-shot examples + a task-specific eval set. Tune only when repeated failures persist and you have representative, high-quality demonstrations.

Always compare candidate model against the **same** baseline and eval data.

Mohammed Vasim

/in/mohd-vasim

PEFT, LoRA & QLoRA

Train fewer parameters without forgetting the math

COMPARISON

Full fine-tune vs PEFT

Method	Mechanism	Trade-off
Full FT	Update all selected model weights.	Best control; very high memory/compute.
LoRA	Learn low-rank weight updates.	Small adapters; limited update rank.
QLoRA	LoRA adapters on a quantized frozen base.	Lower base-weight memory; training compatibility matters.
Prompt/prefix tuning	Train input-level virtual representations.	Very few parameters; variable expressiveness.

FORMULA

LoRA core formula

$$W' = W + (\alpha/r)\cdot BA$$

For $W \in \mathbb{R}^{d_{out} \times d_{in}}$, use $A \in \mathbb{R}^{r \times d_{in}}$, $B \in \mathbb{R}^{d_{out} \times r}$.

Trainable adapter weights: $r(d_{in}+d_{out})$ vs full $d_{out}\, d_{in}$ weights.

Base W stays frozen in standard LoRA.

Hyperparameters to know

Knob	Trade-off
rank r	Capacity vs parameter/memory budget.
alpha α	Adapter update scale, often through α/r.
dropout	Regularizes adapter path.
target modules	E.g., q_proj, v_proj or attention + MLP layers.
LR / epochs	Optimization strength vs overfit risk.
max seq length	Activation/attention cost and task context.

CHECKLIST

Practical workflow

- Format data using the exact model chat template.
- Select a compatible base checkpoint and training precision.
- Mask loss to desired target tokens where appropriate.
- Track train/validation loss and task-level quality.
- Save adapter + base model identity + tokenizer/version.
- Benchmark merged/unmerged serving behavior if relevant.

Symptoms & fixes

Problem	First move
Mem OOM	Lower sequence/batch; checkpoint; QLoRA/sharding.
Worse task accuracy	Audit labels, template, rank, learning rate, data variety.
Good val loss, bad users	Eval mismatch; add representative hard cases.
Weird generation	Confirm EOS/pad behavior and correct chat serialization.

Mohammed Vasim

/in/mohd-vasim

Sampling and decoding

From vocabulary logits to a complete answer

WORKFLOW

Per-step generation

Hidden state → vocab logits → modify logits → sample/select → append token

Stopping happens on EOS, stop strings, max-token limit, or application rules.

Determinism is not always guaranteed even with greedy/low-temperature settings across infrastructure.

FORMULA

Temperature

$$p_i = \exp(z_i/T) / \Sigma_j \exp(z_j/T)$$

Lower T > 0: sharper distribution; **higher T:** flatter distribution.

T → 0: approaches argmax/greedy behavior; systems often special-case exact zero.

Top-k, top-p, greedy & beam

Mode	Effect
Greedy	Pick highest-probability token; low diversity.
Top-k	Sample among k most likely tokens.
Top-p / nucleus	Sample from smallest set with cumulative mass ≥ p.
Beam search	Keep several high-likelihood partial sequences; can favor generic text.

ENGINEERING

Repetition controls

Frequency/presence penalties and no-repeat constraints modify candidate scores. They are implementation-specific; tune against real evals.

Structured output should use schema validation/constrained decoding where available, rather than hoping temperature fixes parsing.

Task → decoding starting point

Task	Good default hypothesis
Extraction / JSON	Constrained schema + low randomness.
Code generation	Low-to-moderate sampling, tests, deterministic verification.
Creative drafting	Higher diversity within safety/quality constraints.
Reasoning task	Task evaluation and adequate output budget; sampling alone is not “reasoning”.

Decoding failures

- Hallucination is not solved by reducing temperature.
- Low temperature may repeat the same incorrect answer.
- Long output increases cost, latency, and exposure to errors.
- Always check stop sequences, context budget, and streaming UX.

> Model decoding controls affect distribution; evidence and verification affect factuality.

Mohammed Vasim

/in/mohd-vasim

Prefill, decode & the KV cache

Why input tokens and output tokens cost differently

LIFECYCLE

Two phases

Phase	Operational meaning
Prefill	Process prompt tokens; build KV cache; comparatively compute intensive.
Decode	Generate one new token at a time; re-use past K/V; often bandwidth/latency limited.

TTFT: request start → first visible token (prefill + scheduling and network).

TPOT: time per output token after first token.

CONCEPTUAL

Caching concept

Without cache: repeat past-token attention projections each step. With cache: append K/V for the new token, attend to existing cache.

Cache usually grows with T until eviction/sliding-window policies apply.

Cache is per active request/sequence; many concurrent users amplify memory.

FORMULAS

Memory estimator

$$\text{Weights} \approx P \times \text{bytes_per_weight}$$

$$\text{KV} \approx 2 \cdot L \cdot B \cdot T \cdot H_{kv} \cdot d_{head} \cdot \text{bytes}$$

Example: a 7B-parameter dense model in 4-bit ideal weight storage is ~3.5 GB before scales, metadata, runtime buffers, and KV cache.

Batching mechanisms

Approach	Purpose
Static batching	Batch known requests together; idle slots can waste throughput.
Continuous batching	Add/remove sequences as tokens complete; improves utilization.
Paged KV cache	Manage KV blocks non-contiguously to reduce fragmentation.
Prefix caching	Reuse eligible shared prompt-prefix work across requests.

Bottleneck diagnosis

Symptom	Investigate
Slow TTFT	Queueing, long prompt, prefill kernel, retrieval/tool latency.
Slow tokens/sec	Memory bandwidth, weak batching, long active contexts.
OOM under load	Concurrent KV cache or buffer growth; admission control.
Latency spikes	Queue depth, oversized requests, cold starts, contention.

Trade-off to remember

> More batch/concurrency → better throughput until capacity saturation, but often worse per-request latency. Optimize under an explicit p95 latency SLO.

Mohammed Vasim

/in/mohd-vasim

Quantization & acceleration

Make serving cheaper without hiding quality costs

FOUNDATIONS

Quantization

Format	Engineering effect
FP32 / BF16 / FP16	Higher precision; larger weight/storage footprint.
INT8	Smaller weights; hardware/kernel dependent speedups.
4-bit weights	Further memory reduction; calibration/format affects error.
KV-cache quantization	Reduce live-context memory; assess long-context quality.

Effective memory includes scale/zero-point metadata and activations; `P×bits/8` is only a lower-bound weight estimate.

IMPLEMENTATION

What FlashAttention changes

Tiles Q/K/V, performs online-normalized attention, and avoids materializing the full `n×n` attention matrix in high-bandwidth memory.

Benefit: reduced IO/memory usage; **not:** linear-time exact full attention.

Actual speedup depends on shapes, dtype, hardware, kernel version.

ACCELERATION

Speculative decoding

A faster draft model proposes tokens; a target model verifies/accepts them under a compatible sampling procedure.

Benefit: fewer expensive target-model sequential steps when acceptance is high.

Cost: draft compute and implementation complexity.

PRACTICAL ORDER

Optimization ladder

- 1. Remove irrelevant input and output tokens.
- 2. Cache safe, repeated prefixes and retrieval results.
- 3. Quantize weights/KV where quality permits.
- 4. Use optimized kernels + continuous batching.
- 5. Pick a smaller model or route simple tasks to one.
- 6. Consider distillation / speculative decoding / architecture changes.

Hardware constraints

Observation	Direction
Weights + KV do not fit	Reduce precision, context/concurrency, or distribute model.
Low GPU utilization	Batch/schedule more effectively; investigate input pipeline.
High bandwidth pressure	Lower precision, fewer KV heads, better caching.
Multi-GPU serving	Parallelize, but account for communication latency.

Measure before/after

Keep constant: model, prompt distribution, output lengths, hardware, concurrency target.

Track: TTFT, TPOT, tokens/sec, p95 latency, GPU memory, cost/task, task success rate.

Speed that breaks answer quality is not an optimization.

Mohammed Vasim

/in/mohd-vasim

RAG vs long context vs fine-tuning

Where knowledge should live

ARCHITECTURE

The baseline RAG loop

Documents → parse/chunk → embed/index

Query → retrieve → rerank → context → answer + citations

Retrieval supplies evidence at request time; it does not alter the generator's weights.

INDEX DESIGN

RAG data contract

- Stable document_id and chunk_id.
- Source URI, title, version, timestamps, access scope.
- Parsed text and optional location/page metadata.
- Embedding model/version, chunking configuration.
- Tenant and ACL filtering before any content reaches the model.

DECISION

Which strategy?

Approach	Best fit
Long context	Occasional analysis of a small set of docs that fit, with acceptable repeated-token cost.
RAG	Large or frequently updated collection; repeated queries; provenance important.
Fine-tuning	Stable response style, formatting, decision pattern, tool behavior.
RAG + FT	Dynamic facts *and* persistent domain behavior are both important.

PRACTICAL

Long-context trade-offs

Side	Reminder
Pros	Simple full-document reasoning; no retrieval infrastructure for a single task.
Cons	Repeated token cost, TTFT, distractors, potential long-context misses.

> A model advertising 1M context does not guarantee reliable retrieval of every fact inside it.

CAUTION

Grounding is conditional

RAG may reduce unsupported claims if it retrieves valid sources and the model uses them correctly.

Failure modes: wrong chunk, missing fact, stale version, untrusted document, ignored evidence.

Require source identifiers and confidence/abstention policies when stakes demand them.

Simple decision tree

Does information change or need a citation? → retrieve or call source tools.

Is output style persistently wrong? → improve prompt and test SFT.

Is request a one-off full-document analysis? → test direct context first.

Is answer an action? → call permissioned tools; generation alone cannot complete it.

Mohammed Vasim

/in/mohd-vasim

Chunking, embeddings & indexing

Retrieval quality begins before search

CHOICES

Chunking methods

Method	Trade-off
Fixed-size	Simple, cheap; may cut meaning.
Recursive / sentence-aware	Respects structure; variable chunk lengths.
Semantic	Split at topical shifts; extra processing.
Hierarchical / parent–child	Retrieve small child; return larger parent.
Document-aware	Use headings, tables, pages, code symbols, metadata.

Overlap: can preserve boundaries, but increases duplicate storage and near-duplicate hits.

MATH

Embedding retrieval

Map query/document chunks into vector space. Compare with cosine similarity or dot product, consistent with embedding model training and normalization.

$$\cos(q,d) = (q \cdot d) / (\|q\|\|d\|)$$

Critical: validate embedding performance on your own domain and language; dimensions alone do not predict quality.

DATABASE

Vector index

Exact nearest neighbor: strong retrieval reference; expensive on large corpora.

ANN: approximate nearest-neighbor index (e.g., HNSW, IVF) trades recall, latency, memory and build/update cost.

Metadata filters, access control, deletion/update behavior are first-class requirements.

RETRIEVAL

BM25 and hybrid search

BM25: sparse lexical scoring using term frequency, rarity, and document-length normalization; excels at exact terms, names, identifiers.

Dense retrieval: semantic similarity; useful for paraphrases.

Hybrid: combine lexical and dense candidates (e.g., reciprocal rank fusion) then rerank.

Use query-type evals: IDs and error codes often need lexical matching.

FAILURE MODES

Document ingestion traps

- Bad PDF extraction merges unrelated columns.
- Tables lose row/column relationships.
- OCR errors alter names, numbers, and formulas.
- Outdated versions conflict with newer policies.
- Missing provenance prevents reliable citations.
- Chunks that cross tenant/security boundaries risk leakage.

Fast quality test

Assemble 30–100 representative queries with known supporting documents. Check **retrieval recall@k** before tuning generation.

If the right evidence is not in retrieved context, prompt tuning is unlikely to fix the core issue.

Mohammed Vasim

/in/mohd-vasim

Reranking & advanced retrieval

Improve precision before generating

PIPELINE

Typical retrieval cascade

Rewrite/query classify → retrieve 20–100 → rerank → select 3–10 → generate

Numbers above are *illustrative starting ranges*, not fixed best practices. Tune by recall, latency and context budget.

COMPARISON

Reranker options

Type	Trade-off
Cross-encoder	Jointly scores query and passage; typically precise but slower.
LLM reranker	Can consider complex instructions; higher latency and cost.
Bi-encoder similarity	Very fast pre-filter; quality depends on embeddings.

When: retrieval recall is adequate but top passages are poorly ordered.

PRE-RETRIEVAL

Query improvement

- **Rewrite:** remove ambiguity / add context, keep original intent.
- **Expansion:** add synonyms or multiple reformulations.
- **Decomposition:** split multi-hop questions into subqueries.
- **Routing:** select keyword, vector, graph, SQL or tool source.
- **Guard:** cap fan-out to contain latency and cost.

PATTERNS

Advanced RAG patterns

Pattern	What it adds
Multi-hop	Retrieve one fact, then use it to retrieve another.
GraphRAG	Traverse explicit entities/relationships and supporting source text.
Adaptive RAG	Route retrieval depth/type based on query complexity.
Self-correction	Evaluate evidence/answer and retry selectively.
Parent–child	Retrieve small unit, supply full parent context.

POST-RETRIEVAL

Ordering & context selection

Remove near-duplicate chunks, prefer authoritative/current sources, maintain adjacency for fragmented evidence, and retain citation metadata.

Use token budgets and explicit evidence thresholds; many lower-quality chunks can hurt the answer.

Test access-control filters independently from semantic relevance.

Debug by stage

Wrong result? ingest/chunk/search. **Right doc, wrong order?** rerank. **Right evidence, wrong answer?** prompt/model/citation verification. **Too slow?** optimize fan-out, caches, and cross-encoder cost.

Mohammed Vasim

/in/mohd-vasim

Tools, agents & MCP

From generating text to taking controlled actions

CONCEPTUAL

What makes something an agent?

LLM workflow: one or more model calls in an application-controlled sequence.

Agent: model can choose a next step/tool based on intermediate state within defined limits.

Core loop: decide → call tool → observe result → continue/stop.

The useful question: what tasks justify dynamic control versus deterministic code?

ARCHITECTURE

Function calling vs MCP

Layer	Responsibility
Function/tool calling	Model outputs a structured request; host executes a concrete function/API.
MCP	Standardizes host/client ↔ server interactions for exposed tools, resources, and prompts.
Agent framework	Controls state, routing, retries, steps, persistence and policy.

> MCP is an integration protocol, not a model or an automatic safety boundary.

INTERFACE

Tool contract example

```
search_docs(query: str, top_k: int) → List[Passage]
```

Provide a clear schema, idempotency expectations, timeouts, auth scope, versioning, and bounded output.

For actions with side effects, explicit confirmations or policy gates may be necessary.

SYSTEMS

Agent state & memory

Scope	Implementation
Short-term	Current messages, task variables, tool outputs.
Long-term	External records or retrieval memory selected with scope and retention.
Durable workflow	Checkpointed state for retries, interruptions and audit.

Never assume text in retrieved memory is trusted instruction.

RISKS

Agent failure modes

- Infinite loops / repeated tool calls.
- Incorrect tool choice or invalid arguments.
- Tool results that contain prompt injection.
- Unauthorized actions from broad permissions.
- Cost and latency amplification from too many steps.
- Unobserved partial failures and duplicate side effects.

Minimum safety controls

Least-privilege tools; typed arguments; schema validation; max steps/time/cost; retries with backoff; idempotency keys; human approval for consequential actions; tool logs and eval tests.

Separate external content from high-priority instructions.

Mohammed Vasim

/in/mohd-vasim

Evaluating the language model

The leaderboard is not your acceptance test

FORMULAS

Loss & perplexity

$$\text{Cross entropy} = -(1/N)\ \Sigma \log p(x_t|x_{<t})$$

$$\text{Perplexity} = \exp(\text{Cross entropy})$$

Lower perplexity indicates better probability assignment on the *same* evaluation distribution and tokenization. It does not directly measure human usefulness.

When reporting, include domain, tokenizer and preprocessing.

MAPPING

Task-specific metrics

Task	Metrics
Extraction	Exact match, field precision/recall/F1, schema validity.
Classification	Accuracy, macro-F1, confusion matrix; calibration where relevant.
Code	Unit tests / pass@k (with clear sampling protocol).
QA	Correctness, groundedness, abstention and citation quality.
Safety	Unsafe compliance / false refusals on curated cases.

PRACTICE

LLM-as-judge

Judge with explicit rubric and ideally blinded comparisons; monitor position/verbosity bias and disagreements with humans.

Useful for scalable qualitative assessment; not an infallible ground truth.

Calibrate on a human-reviewed sample and regularly check drift.

CHECKLIST

Evaluation set design

- Representative production requests and edge cases.
- Hard negatives, ambiguous instructions, adversarial/prompt injection cases.
- Separate development from locked holdout to avoid overfitting.
- Stratify by language, domain, complexity and customer segment.
- Version model, prompts, tools, data and exact scoring rules.

SCOPE

System vs model evaluation

Model: standalone capabilities, factuality, instruction following.

System: prompt + retrieval + tool execution + UI + policies and timeouts.

An agent can fail with a strong model because search, permissions or state handling failed.

Compare candidates correctly

> A/B on the same prompts, retrieval snapshot, generation budget, scoring criteria, and latency/cost envelope. Inspect failures, not just averages.

Report confidence or uncertainty where sample sizes are small.

Mohammed Vasim

/in/mohd-vasim

RAG & agent evaluation

Diagnose the pipeline, not just the final answer

SEARCH

Retrieval metrics

Metric	Interpretation
Recall@k	Fraction of relevant items recovered among top k.
Precision@k	Relevant items divided by k retrieved items.
MRR	Average reciprocal rank of first relevant result.
nDCG@k	Rank-sensitive score using graded relevance.
Hit@k	Did any acceptable source appear in top k?

> Define what counts as a relevant chunk/document before measuring. Retrieval metrics do not guarantee a correct answer.

ANSWER

Generation-side checks

Criterion	Test
Groundedness	Are claims supported by supplied evidence?
Answer correctness	Does response actually solve the question?
Citation accuracy	Do pointers resolve to the claimed passages?
Abstention	Does model decline when evidence is insufficient?
Completeness	Does it cover required subparts without unsupported filler?

Test answer grading with source-aware human review when the application is consequential.

TOOLS

Agent evaluation

- Tool selection accuracy and argument validity.
- Step success, completion rate and user outcome.
- Correct handling of permission-denied, timeout and partial failure.
- Duplicate side-effect rate and rollback behavior.
- Cost, token consumption, number of steps, p95 latency.
- Prompt-injection resistance using malicious tool-return text.

OBSERVABILITY

Error attribution

Question → source exists? → retrieved? → ranked? → used? → answered?

Log intermediate artifacts with privacy controls: rewritten query, top-k IDs, scores, evidence, model/prompt version, tool decisions.

Do not log sensitive raw content unless explicitly allowed and protected.

Regression gate

Block release when a model/prompt/index change materially worsens a critical slice (e.g., privacy, top customer tasks, retrieval recall), even if average score rises.

Rebuild reference answers when source-of-truth policies legitimately change.

Mohammed Vasim

/in/mohd-vasim

Serving architecture & request flow

A minimum dependable production design

SYSTEMS

Reference architecture

Client → API/auth → orchestrator → retrieve/tools → model server → validation → stream

Sidecars/services: prompt registry, experiment tracing, policy checks, metrics, queue, rate limits, cache, data connectors.

Start modular; split into microservices where team, scaling or isolation requirements justify it.

SERVING

Online / async / batch

Mode	Typical choice
Online	Interactive latency SLO; stream responses.
Async jobs	Long runs; queue and notify status.
Batch	Large throughput when latency is secondary.

Use circuit breakers, bounded retries and per-tenant quotas. Keep user-facing and background capacity isolated where needed.

LLMOPS

Data and model versioning

- Pin model weights or API revision where possible.
- Version tokenizer, prompt template and output schemas.
- Version embeddings, vector indexes, reranker, source snapshots.
- Track eval dataset, thresholds, deployment configs.
- Canary and roll back independently of UI releases.

OBSERVABILITY

Capacity metrics

Metric	Meaning
TTFT	Time to first token.
TPOT	Time per generated output token.
TPS	Tokens per second (clarify output vs aggregate).
Throughput	Successful requests/tasks per second.
p95/p99	Tail latency for real traffic.
Utilization	GPU/CPU/memory and queue depth.

APPLICATION

Streaming & UX

Stream when possible, but keep output validation strategy (especially structured data). Provide cancel controls and bounded max output.

Handle disconnects without uncontrolled background generation. Surface citations and tool/action confirmations when relevant.

Operational SLOs

Specify success criteria: **p95 latency ≤ budget**, **quality ≥ threshold**, **cost/task ≤ target**, **error rate ≤ target**.

Monitor by slice: tenant, model, prompt version, query size and tool route.

Mohammed Vasim

/in/mohd-vasim

Security, reliability & governance

Engineering controls that cannot be left to the prompt

SECURITY

Threat map

Risk	Typical vector
Prompt injection	Untrusted docs/tool responses attempt to alter instructions.
Data leakage	Cross-tenant retrieval, logs, responses, tool permissions.
Tool abuse	Excess privileges and unsafe side effects.
Supply chain	Untrusted models, packages, datasets, extensions.
Denial of wallet	Runaway prompts/agent loops/high-cost requests.

CONTROLS

Defense in depth

- Authenticate users; enforce tenant ACLs before retrieval.
- Authorize tools per action, not per model instruction.
- Treat retrieved text and tool outputs as untrusted data.
- Validate structured arguments and outputs.
- Bound context, cost, timeouts and agent steps.
- Redact/minimize sensitive logs; encrypt data and secrets.
- Pen-test with injected documents and malicious tool responses.

GOVERNANCE

Human oversight

Require approval for high-impact, irreversible, legal, financial, or external communication actions according to policy.

Make actions traceable; present real evidence and tool results, not unverified model claims.

SYSTEMS

Reliability patterns

Pattern	Engineering reminder
Retries	Use backoff; avoid duplicate side effects with idempotency.
Fallback model	Verify quality and policy compatibility.
Queue/backpressure	Reject or defer excess load predictably.
Graceful degradation	Answer with limited functionality or safe abstention.
Circuit breaker	Stop repeatedly calling broken dependencies.

DATA

Privacy checklist

Understand each provider's data retention and training options; do not assume policies are identical.

Set data minimization/retention windows; separate production customer data from evaluation/training corpora; handle deletion from caches/indexes/backups.

Test redaction and access boundaries with adversarial fixtures.

Observability boundaries

> Trace request IDs, latency, version hashes, token usage and outcomes. Store raw prompts/tool content only under a documented, access-controlled retention policy.

Review incidents as failures of a *system* (data, prompts, retrieval, tools, model, humans), not merely of a model.

Mohammed Vasim

/in/mohd-vasim

What to change when something fails

Fast diagnosis / real-world decision matrix

TROUBLESHOOT

Problem → highest-value test

Symptom	First diagnostic
Hallucinated policy	Check retrieved source and prompt evidence; enforce abstention.
Low retrieval recall	Evaluate chunking, lexical hybrid, filters and index.
Good recall, bad ranking	Rerank, dedupe, tune candidate depth.
Good evidence, bad response	Improve prompt/instruction, model or SFT; eval claims.
Slow first token	Measure retrieval queue, prompt length, prefill.
Slow generation	Check cache, batch, bandwidth, quantization.
Training OOM	Reduce sequence/microbatch; checkpoint, LoRA/shard.
Agent repeats actions	Add max steps, state inspection, idempotency.
Cross-user leakage	Audit ACL filters, caches, session context immediately.

DECISIONS

RAG vs fine-tuning in one line

Missing/updating facts → retrieval/tools. **Repeated behavior/style error →** SFT/PEFT. **Weak general ability →** model selection or more capable base.

These are hypotheses: choose using a baseline eval and measured costs.

Optimization priorities

Priority	Action
Quality first	Build representative eval and classify failures.
Retrieval second	Ensure evidence reach + ordering.
Latency next	Trim context, efficient serving, queue policy.
Cost next	Right-size model, caching, quantization, batching.
Complexity last	Agents / graph / extra services only when justified.

CHECKLIST

The 6-point production release gate

- 1. Success metrics and held-out test set agreed.
- 2. Security/ACL/privacy tests passing.
- 3. Tail latency at representative concurrency.
- 4. Factuality/citation/abstention thresholds met.
- 5. Cost per successful task within budget.
- 6. Monitoring, rollback and incident owner ready.

A useful mental model

> **Model quality × context quality × orchestration correctness × operational reliability** determines end-user success. Improving one factor cannot compensate for all others.

Mohammed Vasim

/in/mohd-vasim

Formula & complexity crib

Recall the relationships without re-reading chapters

MATH

Must-remember formulas

Concept	Equation
Attention	$\text{softmax}(QK^T/\sqrt{d_k} + \text{mask})V$
Cross entropy	−mean(log p(target token))
Perplexity	exp(token cross entropy)
LoRA update	$W' = W + (\alpha/r)BA$
Cosine similarity	$(a \cdot b)/(\|a\|\|b\|)$
KV cache	$2LBTH_{kv}d_{head}$·bytes
Model weights	params × bytes / parameter
API token cost	(in_tok·in_rate + out_tok·out_rate)/1M

Token prices above assume rates per 1M tokens; include caches and tool costs where applicable.

Parameter and shape reminders

- `B`: batch size; `T`, `n`: tokens/sequence length.
- `L`: transformer layer count; `d`: hidden size.
- `h`: attention query heads; H_{kv}: KV heads.
- `r`: LoRA rank; `P`: parameter count.

COMPLEXITY

Big-O snapshot

Operation	Practical scale
Dense full attention	$O(n^2d)$ attention compute; naïve $O(hn^2)$ score memory.
Local attention (window w)	Often O(nwd) attention compute.
Linear projection	$O(nd^2)$ for dense d×d projection.
Exact vector scan	O(Nd) for N vectors in d dimensions.
ANN search	Index-specific; empirical latency/recall trade-off.
Autoregressive output	Serial dependency across generated tokens.

> Complexities isolate one component and omit batch/layer factors unless shown.

READ THIS

Notation caveats

Attention formulas assume dense dot-product attention. Actual implementations may use GQA, sparse kernels, padding, FlashAttention, cache or specialized hardware.

Quantized formats store extra scales/metadata. Runtime memory exceeds weight and KV lower bounds.

Benchmark on your own length distribution, not a single synthetic prompt.

One-screen retention drill

Can you explain **Q/K/V**, why **KV cache** grows, when **RAG beats long context**, when **LoRA helps**, and how to choose **eval metrics**?

If yes, you can reason about the majority of practical LLM system trade-offs.

Mohammed Vasim

/in/mohd-vasim

Sources & navigation keys

References to your uploaded library; chapter/page locators

PRIMARY: FOUNDATIONS

[R] Build a Large Language Model (From Scratch)

Sebastian Raschka, Manning. Ch 2: tokenization/embeddings (pp 17–49); Ch 3: attention (50–91); Ch 4: GPT block (92–127); Ch 5: pretraining/decoding (128–168); Ch 6–7: classification/SFT (169–250); App E: LoRA (322+).

PRIMARY: SYSTEMS

[C] AI Engineering

Chip Huyen, O'Reilly. Ch 2: foundation models; Ch 3–4: evaluation; Ch 5: prompting/security; Ch 6: RAG/agents; Ch 7: fine-tuning; Ch 8: dataset engineering; Ch 9: inference optimization.

PRIMARY: ENGINEERING

[H] LLM Engineer's Handbook

Paul Iusztin & Maxime Labonne, Packt. Ch 1: FTI architecture; Ch 4 / 9: RAG; Ch 7–8: fine-tuning/evaluation; Ch 10: deployment; Ch 11: LLMOps.

SUPPLEMENT

[P] LLMs in Production

Christopher Brousseau & Matthew Sharp, Manning. Ch 2: language modeling; Ch 3: LLMOps; Ch 4–5: data/training; Ch 6: serving; Ch 7–8: prompting/apps.

PRIMARY: RAG

[G] Mastering RAG

Pratik Bhavsar / Galileo, v1.01. Ch 1: RAG vs long context; Ch 2: enterprise design; Ch 3: chunking; Ch 4: embeddings; Ch 5: vector DBs; Ch 6: reranking; Ch 7: production testing.

SUPPLEMENT

[F] Hands-On Guide to Fine-Tuning LLMs

Daniel Voigt Godoy. Ch 0: practical QLoRA/SFT workflow; later chapters: fine-tuning and adaptation with PyTorch and Hugging Face.

SUPPLEMENT

[M] MCP: The Illustrated Guidebook

Avi Chawla & Akshay Pachaar. Section 1 (pp 3–18): MCP, client/server roles, tools/resources/prompts. Section 2: integration projects.

SUPPLEMENT

[A] Architecting AI Software Systems · [D] DDIA 2e

[A] Richard D Avila & Imran Ahmad: AI architectures, microservices, nonfunctional requirements, governance. **[D] Martin Kleppmann & Chris Riccomini**: reliable/scalable data systems, replication and system trade-offs.

METHOD

How sources were used

Explanations are **original concise syntheses**, not copied pages. Chapter/page pointers refer to the publications' printed chapter pages where shown.

Formula and operational examples are standard engineering derivations/illustrations; numerical examples are marked as such.

Vendor-specific model limits, prices and performance change rapidly and were intentionally omitted.

Mohammed Vasim

/in/mohd-vasim